

ML Specification Report — playground-series-s3e19

Forecast Mini-Course Sales · our from-scratch agent (GBDT pool + tree-search v3)

Figures grounded in the competition's `facts.json`, `eda_summary.json`, `STATUS.md`, and `experiments_tree_v3.json`.

Contents

• Forecast Mini-Course Sales · our from-scratch agent (GBDT pool + tree-search v3)	1
• Overview	1
• Data	1
• Models & Architecture	3
• Training procedures	3
• Inference procedures	4
• Evaluation & Benchmarking	4
• Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08	5

Overview

The task is to forecast daily unit sales (`num_sold`) for 75 mini-course series across all of 2022 (metric: SMAPE, minimize). We solve it with a from-scratch pipeline — a pool of gradient-boosted trees (LightGBM / XGBoost / CatBoost) on a `log1p` target, refined through a four-round self-improvement ablation and a 22-node tree search. The single methodological decision that dominates everything here is the **cross-validation scheme**: because the test period (2022) lies strictly *after* the training period (2017–2021), we validate with **TimeSeriesSplit** — forcing every fold to extrapolate into unseen future dates, exactly as the real submission must. Our benchmarked best is **CV SMAPE 10.019** under that honest scheme; the tree search pushed the OOF number further to 9.757. The NVIDIA reproduce-agent, replaying a public LightGBM kernel's technique on the *identical* hard TimeSeriesSplit-5fold, reaches only 13.72 — **our agent wins by a wide margin**, and the reason is precisely that we chose the task-correct CV instead of an optimistic random split.

Why it matters. Reliable sales forecasting drives inventory, staffing, and revenue planning; as a time-extrapolation task it also tests whether validation respects temporal ordering.

Data

Purpose of Data. Forecast the number of units sold per day for every (date × country × store × product) series — a tabular time-series regression Playground Series episode. **Data Format** is clean **tabular CSV** with four raw predictive columns (`date`, `country`, `store`, `product`) once `id` and the target `num_sold`

are removed. **Data Volume** is **136,950 training rows (2017-01-01 → 2021-12-31, 1,826 days) / 27,375 test rows (all of 2022, 365 days)**, arranged as **75 complete daily series** (5 countries × 3 stores × 5 products); test rows = 75 × 365.

Data Quality is high: **no missing values** in either split, **0 duplicate rows**, every series complete daily. The target is right-skewed (**skew 1.747**, mean 165.5, median 98, max 1,380), which becomes near-symmetric under `log1p` (≈ -0.22) — so all learners train on `log1p(num_sold)` and invert with `expm1` before scoring. The strongest EDA signals, all consistent with a multiplicative sales structure, were:

- **Strong multiplicative structure:** an additive OLS of `log1p(num_sold)` on country + store + product alone (no date) explains $R^2 = 0.974$ of target variance — the categoricals carry most of the signal.
- **Store/product shares are rock-stable** across years (Kagglazon ≈ 0.691 of the yearly total every year; product shares drift $< \pm 0.005$), whereas **country shares drift** (Argentina 0.0999 → 0.0663; Canada 0.3052 → 0.3244) — the country × time interaction is the hardest part of the extrapolation.
- **Seasonality:** weekend uplift (Sunday mean 188.4 vs Mon–Thu ≈ 157), a Dec/Jan peak (Dec 183.5, Jan 175.7 vs Apr 156.9), and a Jan-1 spike (216.7 vs the 165.5 overall mean).
- **Yearly totals are non-monotonic** (2020 COVID-style dip 4.09 M, 2021 rebound 4.88 M) — there is no reliable linear trend to extrapolate, which later sank the ratio-decomposition experiment.

Annotation Guidelines. The label is `num_sold` $\in \mathbb{Z}^+$ (integer daily units). Submissions are **continuous sales forecasts** scored by SMAPE — a scale-relative regression error, not a classification.

Feature Set. From the four raw columns we engineer **20 features**, grouped below:

Group	Features
Calendar	<code>year</code> , <code>month</code> , <code>day</code> , <code>dow</code> , <code>day_of_year</code> , <code>weekofyear</code> , <code>quarter</code>
Flags	<code>is_weekend</code> , <code>is_month_start</code> , <code>is_month_end</code> , <code>is_new_year</code>
Cyclical	<code>month_sin</code> , <code>month_cos</code> , <code>dow_sin</code> , <code>dow_cos</code> , <code>doy_sin</code> , <code>doy_cos</code>
Categoricals	<code>country_cat</code> , <code>store_cat</code> , <code>product_cat</code> (label-encoded, fed natively as categoricals to all three models)

Splitting strategy. This is the decisive methodology choice. The test set (2022) is entirely *after* the training span (2017–2021) with **zero date overlap**, so the honest yardstick is **TimeSeriesSplit(n_splits = 5) on the 1,826 unique dates** (expanding window) — every validation block is strictly later than its training window, mirroring the real train → test gap. A reflexion diagnostic proves this matters: the *same* 20 features and 3 models score **SMAPE 4.281 under a random shuffled 5-fold KFold** but **10.175 under TimeSeriesSplit** — not a regression, but the difference between an optimistic interpolation score and an honest extrapolation score. The rule captured here is *never compare scores across CV schemes*, and every experiment logs its `cv.strategy`. TimeSeriesSplit is deterministic so needs no fold seed; the shuffled diagnostic used seed 42, and a dedicated `cv_seed` field is **not recorded**. The leaderboard was **not** used (unattended batch run, no credentials), so all scores here are **CV-only**.

Models & Architecture

Purpose of Architecture. A daily-sales regressor that minimizes SMAPE under future-year extrapolation. **Architecture Type** is a **gradient-boosted decision-tree ensemble** — LightGBM as the primary learner (it dominates every fold under extrapolation), with CatBoost members and a `log1p` target transform; XGBoost was in the initial pool but was weight-searched to 0 and dropped. The linear-iteration champion is a **6-member weighted blend** (experiment #7); the tree-search champion is a **10-member blend with an OOF-fitted global multiplier**, discovered at tree-search node #17.

Each member takes the same **Input Format** — a numeric feature matrix with the three categoricals label-encoded and fed natively as categoricals — of **Input Dimension 20 features** per row (a 1-D vector; no spatial/sequence tensor structure; the temporal information is encoded as calendar/cyclical features).

Architecture Description. Each GBDT member is an additive ensemble of decision trees grown by histogram-based, leaf-wise gradient boosting on the `log1p` target (representative hand-set LGB: `n_estimators` = 2000, `learning_rate` = 0.03, `num_leaves` = 63; the Optuna-tuned LGB is shallower — `num_leaves` = 20, `min_child_samples` = 38 — consistent with "extrapolation rewards regularization"). A blend layer sits on top as a weighted average of members' out-of-fold predictions, with weights found by grid search (linear rounds) or Dirichlet sampling (tree search); the tree-search champion additionally applies a single grid-searched **global $\times 1.02$ multiplier** correcting a systematic ~2% low bias in TimeSeriesSplit OOF. **Model Complexity** is best expressed as trees $\times$ leaves rather than a dense parameter count: a single LGB member $\approx$ 2000 iterations $\times$ tens of leaves, and the champion blends up to 10 such members.

Training procedures

Training proceeds as a **four-round self-improvement ablation** on the shared TimeSeriesSplit-5fold split, each round adding capability on the same OOF scale, followed by a tree-search layer:

Round / layer	Configuration	TimeSeriesSplit blend SMAPE
baseline (exp #2)	LGB+XGB+CAT, 20 features, <code>log1p</code> target, grid-blend	10.17540
+seed bag (exp #5)	+ LGB seed 2024	10.16605
+extended seed bag (exp #6)	+ LGB seed 7, + CAT seed 2024	10.15721
+Optuna-tuned LGB (exp #7)	+ fold-5-proxy tuned LGB, 6-way blend	10.01946
tree search v3 (node #17)	10-way blend + <code>auto_scale</code> $\times 1.02$	9.75707

(The random-KFold baseline exp #1 scored 5.31891 and the same-feature diagnostic exp #3 scored 4.28142 — reported for the CV-scheme comparison only, not comparable to the TimeSeriesSplit numbers above.)

The **Loss Function** is L2 regression on the `log1p` target (LightGBM regression objective), with model selection on SMAPE after `expm1` inversion; the ratio-decomposition member (harmonic-linear structural model) was ablated and *rejected* (solo SMAPE 14.75, weight-searched to 0 — the structural signal does not beat GBDT here, unlike s3e20). The **Optimization Algorithm** is gradient boosting (histogram, leaf-wise) — **not** SGD/ADAM — with blend weights optimized by grid search / Dirichlet sampling. The **Learning Rate**

is the boosting shrinkage (hand-set LGB `learning_rate` = 0.03; Optuna-tuned = 0.0371); there is **no Learning Rate Scheduler** (*N/A — GBDT convergence is governed by iterations, not a schedule*) and **no Batch Size** (*N/A — full-dataset histogram boosting, not mini-batched*).

Training Duration for the tree-search layer was **321.0 s of wall-clock over 22 evaluated nodes** (CatBoost seeds dominate the budget; every LGB solo ≤ 7 s), and the Optuna tune was **40 trials / 42.0 s**; the run performed **22 model fits** in total. **Training Memory** was not explicitly capped — a $136,950 \times 20$ float matrix is trivial on the single arm64 CPU machine, peak **not recorded**. **Transfer Learning** is *N/A (no pretrained weights)*; its analogue here is **cross-competition experience injection** — the seed-bagging pattern distilled from prior episodes (s3e9/s3e14) carried in via `experience.md`. **Data Augmentation** is *N/A (tabular)*; the analogues are feature engineering (the 20 calendar / cyclical features) and seed-bagging (LGB seeds 42/2024/7/3000, CAT seeds 42/2024).

Reproducibility Standards are strict: fixed seeds (model seeds 42/2024/7/3000; Optuna seed logged), LightGBM determinism flags, and TimeSeriesSplit fold construction verified digit-for-digit against the training pipeline before searching. A **faithful replay reproduced 20 of 22 nodes exactly, with the champion node $\Delta = 0$** (the two inexact nodes are early-batch LGBM nondeterminism, not the champion).

Inference procedures

Decision Threshold is *N/A* — SMAPE is a regression forecast metric, so we submit continuous `expm1`-inverted sales predictions clipped at 0 and never threshold (a cut-point would only apply to a classification task). **Inference Duration** was not separately profiled; GBDT scoring of the 27,375 test rows is sub-second per member on CPU, and **Inference Memory** is negligible (tree inference is lightweight, not capped). Note the tree-search champion (node #17) is an **OOF-only** search result — no test predictions were generated for it, so no submission file exists; the submitted best is the `exp #7 6-way blend (sub_6way_optuna_blend_10.01946_...csv)`.

Evaluation & Benchmarking

Task for Performance Evaluation. Forecast 2022 daily unit sales for all 75 series. **Performance Metrics.** SMAPE is the sole competition metric (minimize); our benchmarked best is **CV OOF SMAPE 10.019** (exp #7, the 6-way Optuna-tuned blend), improving $10.17540 \rightarrow 10.16605 \rightarrow 10.15721 \rightarrow 10.01946$ across the four self-improvement rounds, all under the identical TimeSeriesSplit-5fold scheme. The tree-search layer pushed the OOF number to **9.75707** (node #17), but that figure carries fold-5 double-dip optimism plus an OOF-fitted scale/seed selection — its honest read is *"meaningfully below 10.02, not literally 9.76"* — so **10.02 is the conservative, apples-to-apples number** we benchmark. No leaderboard score exists (CV-only run).

Performance Benchmarking. The comparator is the **NVIDIA reproduce-agent**, which replays the technique of the strongest public kernel:

Agent	Approach	TimeSeriesSplit-5fold SMAPE	Note
Our agent	from-scratch GBDT pool + tree search	10.02	winner

Agent	Approach	TimeSeriesSplit-5fold SMAPE	Note
NVIDIA	reproduces <code>tumpanjawat/s3e19-course-eda-fe-lightgbm</code> (technique)	13.72	-3.7 SMAPE

Our agent wins decisively (10.02 vs 13.72, lower is better) — the widest margin of any comp in this sweep. The honest framing: *both* numbers are scored on the **identical hard TimeSeriesSplit-5fold** scheme, so this is a like-for-like comparison, and the gap is attributable to methodology rather than luck. Our edge is exactly the **task-correct CV choice** — we recognized that 2022 is a pure future-year extrapolation and validated accordingly, which drove the whole pipeline (regularized tuning, seed-bagging to cut the inflated fold variance, the $\times 1.02$ level correction) toward what actually generalizes forward, whereas a kernel optimized on an optimistic random split transfers poorly to the honest extrapolation test. On this time-series episode, choosing the right validation scheme *is* the win.

Final-Architecture Re-Run (v5 pipeline, Stage 0.5 in-lane) — 2026-08

All 20 baseline competitions were re-run in an isolated root (`~/benchruns/myagent-rerun`) under one frozen architecture (`docs/rerun_manifest.json`): every lane runs the **full v5 pipeline including the Stage 0.5 problem dossier** (`dossier.json` written in-lane before modeling), holds no credentials, and is checked by a per-lane transcript audit. Leaderboard scores for the re-run were produced by the separate credentialed operator scoring step and are recorded in `benchmark_results/rerun_three_way.csv` ; the figures below revise nothing in the sections above.

- **Lane:** done 2026-08-17, 39 min — clean re-run; the 08-14 first attempt was discarded by the transcript audit (raw knowledge-library opens + a network call), and the clean run scores worse on CV (6.706 vs the tainted 5.740), consistent with the audit pricing a real contamination advantage; the re-run's own audit flag was a verified detector false positive (operator waiver on record).
- re-run local CV: **pooled OOF SMAPE 6.70585** (3 expanding year-forward folds), structural Ridge in log space on the `ratio_target` arm (node #17).
- Public / private leaderboard for this re-run (`benchmark_results/rerun_three_way.csv`): **Public 49.11872 / Private 49.40878**; `winner_priv` = **nvidia** (NVIDIA 48.26558, AIDE 48.34131). Note the reversal against the CV headline above: our decisive local-CV win does **not** carry to the private leaderboard, where NVIDIA is ahead.